Let’s combine it with what we have covered so far. The goal of this example is to read a file with the following content

```
name,surname,age
Alice,Pleasence,19
Bob,Marley,30
Benjamin,Button,2
```

and filter out people younger than 18, resulting in the following:

```
Alice,Pleasence,19
Bob,Marley,30
```

To write the result to a file, you must format the filtered rows back into a single `ByteString`. You can do this with the `CsvFormatting.format()` method. With this command and `FileIO` to read from and write to, you have everything you need.

When reading a file with `FileIO.fromPath(f: Path): ByteString`, you get the contents as a single `ByteString`. You also need a single `ByteString` to write to a file with `.toPath`, but in between, you have to convert the `ByteString` into something you can filter and convert it back to a `ByteString` with `CsvFormatting.format()`.

Listing 14.9 Reading and writing a file

```
import akka.stream.scaladsl.FileIO
import java.nio.file.{ FileSystems, Paths }

object CSVSpec {
  ...

  "A CSV file" should "be able to be read and write another CSV file" in {

    val fs = FileSystems.getDefault
```